Merge Conflict
Programmer's Manual
CS450 – Spring 2025

Table of Contents

C Files

comexec.c

- Comexec.c is where the execution of all user entered commands occurs. It holds functionality for version, help, get_time, get_date, set_time, set_date, and shutdown.

comexec()

- int comexec(char buf[])

- Authors: Tanner Forbes, Chris Jones, Izaak Whetsell, Evan Humphrey

- This function executes the different system commands after reading the buffer in from comhand()

- Parameters:
  - char buf[] is an array of characters that the user has typed

- Return Values:
  - This function returns an integer that represents the status of the functions completion.
  - 1 is to shutdown the operating system
  - 0 is to continue running

shutdown()

- int shutdown(void)

- Authors: Izaak Whetsell, Chris Jones

- This function terminates the command handling sequence

- Parameters:
  - None

- Return Values:
    - This function returns an integer that represents the status of the functions completion.
    - 1 is to shutdown the operating system
    - 0 is to continue running

help()

- int help(void)

- Author: Chris Jones

- This function prints a list of each available command

- Parameters:
    - None

- Return Values:
    - This function returns an integer that represents the status of the functions completion.
    - 1 is to shutdown the operating system
    - 0 is to continue running

version()

- int version(void)

- Authors: Chris Jones, Evan Humphrey

- This function prints the current version of the system

- Parameters:
    - None

- Return Values:

        - This function returns an integer that represents the status of the functions completion.
        - 1 is to shutdown the operating system
        - 0 is to continue running

clear_screen()
  - int clear_screen(void)

  - Authors: Evan Humphrey

  - This function clears the window being viewed in the terminal and returns the cursor to the “home” position

  - Parameters:
      - None

  - Return Values:
      - This function returns an integer that represents the status of the functions completion.
      - 0 for success

return_color()
  - char* return_color(void)

  - Authors: Chris Jones

  - This function returns the desired color code saved into a special variable

  - Parameters:
      - None

  - Return Values:
      - This function returns a string that holds the color code requested

include/mpx/comexec.h

This file contains all functions that execute system commands when called in the terminal

Function Prototypes:
int comexec(char buf[])

- **This function is called to execute any command called in the system, it will scan through the input and determine whether a command is called correctly or incorrectly**

int shutdown(void)

- **This function will terminate the command handling sequence in the system and shut down any processes executing**

int help(void)

- **This function will print a list of each available command from the current point in the system**

int version(void)

- **This function prints the compilation date and version of the system**

int clear_screen(void)

- **This function clears the screen that you are viewing in the terminal window and returns the cursor to the “home” position**

char* return_color(void)

- **This function returns the desired color requested by a special variable in the program**

include/string.h

This file contains all functions pertaining to string manipulation and identification

int contains(const char *str1, const char *str2)

- **This function indicates whether str1 contains str2**

char* substr(char* str1, int index)

- **This function will return the substring of a passed-in string beginning at a starting index**

int isNumeric(char* str)

- **This function determines whether a string is entirely numeric**

int charCount(char* str, char c)

- **This function finds the number of occurrences of a specific character in a given string**

int indexOf(char* str, char searched_char)

- **This function finds the specific index of a desired character in a string**

void print(const char* sentence)

- **This function prints out the given sentence into the terminal window**

void println(void)

- This function prints a newline to the terminal

void print_error(const char* sentence)
- This function prints an error into the terminal window, this means that the text is printed out in a red color

void print_color(const char* sentence, const char* color_code)
- This function prints out text into the terminal into a variety of colors

void trim(char sentence[])
- This function trims the whitespace off the ends of the given string

include/mpx/serial.h

This header file contains the functions and constants used in serial_poll()

Constants include:
- UP_ARROW
- DOWN_ARROW
- LEFT_ARROW
- RIGHT_ARROW
- DELETE_KEY
- BACKSPACE
- NEW_LINE
- CARRAIGE_RETURN
- ESCAPE_SEQUENCE

All of which represent the character sequence of each of their respective keys.

Function Prototypes:
int serial_init(device dev)

- This function initializes the devices connected to the OS for input and output

int serial_out(device dev, const char *buffer, size_t len)

- This function writes the contents of the buffer to the serial port

int serial_poll(device dev, char *buffer, size_t len)

- This function reads the contents of a string to the serial port

void buffer_refresh(char *buffer, int buf_length, int pos)

- This function refreshes the buffer in the terminal window after a key is pressed

include/mpx/device.h

This header file contains the constants used to identify the device I/O ports in the system

Enums:
device
COM1,
COM2,
COM3,
COM4

Each of these constants contain a memory address which gives the location of the I/O ports to be written to/read from

include/sys_req.h

This file contains all system request functions and constants

Enums:
op_code
EXIT,
IDLE,
READ,
WRITE

This enumeration defines the constants needed to make a system request to perform a specific action

Constants:

- **INVALID_OPERATION**
- **INVALID_BUFFER**
- **INVALID_COUNT**

These constants define some typical errors and return values that might arise with system calls.

Function Prototypes:
int sys_req(op_code op, ...)

- This function requests a kernel operation within the system to perform an action based on the input parameters.

include/mpx/io.h

This header file contains kernel macros to write and read to I/O ports

Function Prototypes:

outb(port, data)

- **This will take one byte of data and write it to an I/O port**

inb(port, data)

- **This will take one byte of data and read it into an I/O port**

include/mpx/interrupts.h

This file contains kernel functions related to software and hardware interrupts in the system

Constants include:

- **cli()**
- **sti()**

These constants are used to disable and enable interrupts in the system respectively

Function Prototypes:

void irq_init(void)

- This function installs the first 32 interrupt handlers into the IRQ (Interrupt Request Table)

void pic_init(void)

- This function initializes the programmable interrupt controllers and performs the necessary remapping of IRQs, it leaves interrupts turned off.

void idt_init(void)

- This function creates and installs the interrupt descriptor table

void idt_install(int vector, void (*handler)(void *))

- The function installs an interrupt handler

include/stdlib.h

This file contains a subset of C standard functions

Function Prototypes:
int atoi(const char *s)

- This function converts an ASCII string to an integer literal

char* itoa(int num, char* str, int base)

- This function converts a given integer value into an ASCII string

char bcdToChar(unsigned char bcd)

- This function converts a Binary Coded Decimal Value (BCD) into a character

unsigned int intToBCD(unsigned int num)

- This function converts a character into a Binary Coded Decimal Value (BCD)

include/ctype.h

This file contains a subset of C standard function

Function Prototypes:
int isspace(int c)

- **This function determines if the given character is a whitespace**

int isdigit(char c)

- **This function determines of a given character is a digit**

include/memory.h

This file contains system-specific dynamic memory functions

Function Prototypes:
void *sys_alloc_mem(size_t size)

- **This function dynamically allocates memory in the system**

int sys_free_mem(void *ptr)

- **This function frees dynamically allocated memory located within the system**

void sys_set_heap_functions(void * (*alloc_fn)(size_t), int (*free_fn)(void *))

- **This function installs user-supplied heap-management functions**

include/mpx/time.h

This file contains all functions relating to the real time clock (RTC)

Function Prototypes:

int get_time(void)
- This function prints out the current time in Eastern Standard Time using the real time clock

int set_time(char buf[])
- This function utilizes the real time clock registers to set the time in the system

int get_date(void)
- This function retrieves the date using the real time clock and prints it to the terminal

int set_date(char buf[])
- This function utilizes the real time clock registers to set the current date within the system

int get_hour(void)
- This function returns an integer representation of the current hour in the system.

int get_minute(void)
- This function returns an integer representation of the current minute in the system.

int get_second(void)
- This function returns an integer representation of the current second in the system.

include/mpx/user_pcb.h

This file contains all user functions related to the creation, alteration, deletion, and display of all process control blocks.

Function Prototypes:

int create_PCB(char* name, int class, int priority)
- This function creates a new process and assigns values into it based on its given parameters

int delete_PCB(char* name)
- This function deletes a PCB from memory

int block_PCB(char* name)
- This function edits a given process to be put into a blocked state

int unblock_PCB(char* name)
- This function edits a given process to be placed into a ready state

int suspend_PCB(char* name)
- This function edits a given process to be placed into a suspended state

int resume_PCB(char* name)
- This function edits a given process to be placed into a not suspended state

int set_PCB_priority(char* name, int priority)
- This function takes a priority and edits a given process to change to the specified priority

int show_PCB(char* name)

- This function displays a given process block and its information given its name

int show_ready_PCB(void)
- This function displays all ready processes and their associated information into the terminal window

int show_blocked_PCB(void)
- This function displays all blocked processes and their associated information into the terminal window

int show_all_PCB(void)
- This function displays all existing processes from the system into the terminal window

int Load_R3(void)
- This function deploys 5 new processes to test the dispatching method of the R3 module implementation.

int Load_R3_Sus(int pcb_num, int priority)
- This function loads a ready, suspended process into the system with a numbered name specified in the function.

includes/mpx/sys_call.h

This file contains all functions pertaining to process dispatching.

Global Variables:
pcb* CURRENT_PCB

This pointer points to the currently running process in the system, and is updated during every context switch.

ctx* GLOBAL_CTX

This pointer points to the context of the initial process loaded in the system, and is used to return back to the system's initial state when an *EXIT* is issued.

pcb* nextPCB

This pointer is used to track the position of the next process in the queue.

struct context* sys_call(context* ctx)

- This function returns a pointer to the next process to execute in the system.

pcb* find_first_ready(void)

- This functions finds and returns the first process in the ready queue.

includes/mpx/alarm.h

This file contains all functions pertaining to alarm creation and management.

void alarm_process(void)

- This function monitors the alarm processes that are contained within the ready queue.

int create_alarm(char* message, char* time)

- This function takes the parameters needed from the user and creates an alarm process to be inserted into the queue.

comhand.c

- This file contains the command handler, which scans for any commands entered into the system

comhand()

- void comhand(void)

- Authors: Izaak Whetsell

- This function scans for commands to execute in the system which passes them on to be executed in the system

- Parameters:
  - None

- Return Values:
  - None

include/mpx/comexec.h

This file contains all functions that execute system commands when called in the terminal

Function Prototypes:
int comexec(char buf[])

- This function is called to execute any command called in the system, it will scan through the input and determine whether a command is called correctly or incorrectly

int shutdown(void)

- This function will terminate the command handling sequence in the system and shut down any processes executing

int help(void)

- **This function will print a list of each available command from the current point in the system**

int version(void)

- **This function prints the compilation date and version of the system**

int clear_screen(void)

- **This function clears the screen that you are viewing in the terminal window and returns the cursor to the “home” position**

char* return_color(void)

- **This function returns the desired color requested by a special variable in the program**

include/mpx/device.h

This header file contains the constants used to identify the device I/O ports in the system

Enums:
device
COM1,
COM2,
COM3,
COM4

Each of these constants contain a memory address which gives the location of the I/O ports to be written to/read from

include/mpx/serial.h

This header file contains the functions and constants used in serial_poll()

Constants include:

- UP_ARROW
- DOWN_ARROW
- LEFT_ARROW
- RIGHT_ARROW
- DELETE_KEY
- BACKSPACE
- NEW_LINE
- CARRAIGE_RETURN
- ESCAPE_SEQUENCE

All of which represent the character sequence of each of their respective keys.

Function Prototypes:

int serial_init(device dev)

- This function initializes the devices connected to the OS for input and output

int serial_out(device dev, const char *buffer, size_t len)

- This function writes the contents of the buffer to the serial port

int serial_poll(device dev, char *buffer, size_t len)

- This function reads the contents of a string to the serial port

void buffer_refresh(char *buffer, int buf_length, int pos)

- **This function refreshes the buffer in the terminal window after a key is pressed**

include/sys_req.h

This file contains all system request functions and constants

Enums:
op_code
EXIT,
IDLE,
READ,
WRITE

This enumeration defines the constants needed to make a system request to perform a specific action

Constants:

- **INVALID_OPERATION**
- **INVALID_BUFFER**
- **INVALID_COUNT**

These constants define some typical errors and return values that might arise with system calls.

Function Prototypes:
int sys_req(op_code op, ...)

- **This function requests a kernel operation within the system to perform an action based on the input parameters.**

include/string.h

This file contains all functions pertaining to string manipulation and identification

Function Prototypes:
void* memcpy(void * restrict dst, const void * restrict src, size_t n)

- This function will copy a desired region of memory to a destination region in memory and will return a pointer to the destination region

void* memset(void *address, int c, size_t n)

- This function will take a region of memory and fill it with a desired amount of memory and return a pointer to that memory

int strcmp(const char *s1, const char *s2)

- This function compares two strings and returns an integer code specifying the result of the operation

size_t strlen(const char *s)

- This function returns the length of a passed-in string

char* strtok(char * restrict s1, const char * restrict s2)

- This function will tokenize a string and return

int contains(const char *str1, const char *str2)

- This function indicates whether str1 contains str2

char* substr(char* str1, int index)

- This function will return the substring of a passed-in string beginning at a starting index

int isNumeric(char* str)

- This function determines whether a string is entirely numeric

int charCount(char* str, char c)

- This function finds the number of occurrences of a specific character in a given string

int indexOf(char* str, char searched_char)

- This function finds the specific index of a desired character in a string

void print(const char* sentence)

- This function prints out the given sentence into the terminal window

void println(void)

- This function prints a newline to the terminal

void print_error(const char* sentence)

- This function prints an error into the terminal window, this means that the text is printed out in a red color

void print_color(const char* sentence, const char* color_code)

- This function prints out text into the terminal into a variety of colors

`void trim(char sentence[])`

- This function trims the whitespace off the ends of the given string

ctype.c

- ctype.c holds functions relating to the manipulation or extraction of data of the char type.

isdigit()

- int isdigit(char c)

- Author: Chris Jones

- This function determines whether a given character is a digit

- Parameters:
  - char c is the character that is being judged to be or not to be an integer

- Return Values:
  - This function returns an int that represents a boolean.
  - 1 is true and the character is a digit
  - 0 is false and the character is not a digit

include/ctype.h

This file contains a subset of C standard function

Function Prototypes:
int isdigit(char c)

- This function determines of a given character is a numeric digit and returns an integer code of 1 if the character is a digit, and a 0 if not

serial.c

- serial.c holds methods and structures pertaining to the input and output of serial communications.

buffer_refresh()
void buffer_refresh(char *buffer, int buf_length, int pos)

- Author: Evan Humphrey

- This function keeps track of the buffer in the terminal window as keys are pressed, refreshing every time the buffer is updated.

- Parameters:
  - char *buffer is a string of current characters typed by the user
  - int buf_length is the length of the passed buffer
  - int pos is the position of the user's cursor

- Return Values:
  - None

include/serial.h

This header file contains the functions and constants used in serial_poll()

Constants include:

- UP_ARROW
- DOWN_ARROW
- LEFT_ARROW
- RIGHT_ARROW
- DELETE_KEY

- BACKSPACE
- NEW_LINE
- CARRAIGE_RETURN
- ESCAPE_SEQUENCE

All of which represent the character sequence of each of their respective keys.

Function Prototypes:

int serial_init(device dev)

- This function initializes the devices connected to the OS for input and output

int serial_out(device dev, const char *buffer, size_t len)

- This function writes the contents of the buffer to the serial port

int serial_poll(device dev, char *buffer, size_t len)

- This function reads the contents of a string to the serial port

void buffer_refresh(char *buffer, int buf_length, int pos)

- This function refreshes the buffer in the terminal window after a key is pressed

include/mpx/io.h

This header file contains kernel macros to write and read to I/O ports

Function Prototypes:

outb(port, data)

- **This will take one byte of data and write it to an I/O port**

inb(port, data)

- **This will take one byte of data and read it into an I/O port**

include/string.h

This file contains all functions pertaining to string manipulation and identification

Function Prototypes:
void* memcpy(void * restrict dst, const void * restrict src, size_t n)

- **This function will copy a desired region of memory to a destination region in memory and will return a pointer to the destination region**

void* memset(void *address, int c, size_t n)

- **This function will take a region of memory and fill it with a desired amount of memory and return a pointer to that memory**

int strcmp(const char *s1, const char *s2)

- **This function compares two strings and returns an integer code specifying the result of the operation**

size_t strlen(const char *s)

- This function returns the length of a passed-in string

char* strtok(char * restrict s1, const char * restrict s2)

- This function will tokenize a string and return

int contains(const char *str1, const char *str2)

- This function indicates whether str1 contains str2

char* substr(char* str1, int index)

- This function will return the substring of a passed-in string beginning at a starting index

int isNumeric(char* str)

- This function determines whether a string is entirely numeric

int charCount(char* str, char c)

- This function finds the number of occurrences of a specific character in a given string

int indexOf(char* str, char searched_char)

- This function finds the specific index of a desired character in a string

void print(const char* sentence)

- This function prints out the given sentence into the terminal window

void println(void)

- **This function prints a newline to the terminal**

void print_error(const char* sentence)

- **This function prints an error into the terminal window, this means that the text is printed out in a red color**

void print_color(const char* sentence, const char* color_code)

- **This function prints out text into the terminal into a variety of colors**

void trim(char sentence[])

- This function trims the whitespace off the ends of the given string

include/sys_req.h

This file contains all system request functions and constants

Enums:
op_code
EXIT,
IDLE,
READ,
WRITE

This enumeration defines the constants needed to make a system request to perform a specific action

Constants:

- **INVALID_OPERATION**
- **INVALID_BUFFER**
- **INVALID_COUNT**

These constants define some typical errors and return values that might arise with system calls.

Function Prototypes:
int sys_req(op_code op, ...)

- This function requests a kernel operation within the system to perform an action based on the input parameters.

include/stdlib.h

This file contains a subset of C standard functions

Function Prototypes:
int atoi(const char *s)

- This function converts an ASCII string to an integer literal

char* itoa(int num, char* str, int base)

- This function converts a given integer value into an ASCII string

char bcdToChar(unsigned char bcd)

- This function converts a Binary Coded Decimal Value (BCD) into a character

unsigned int intToBCD(unsigned int num)

- This function converts a character into a Binary Coded Decimal Value (BCD)

stdlib.c

- Stdlib.c contains all functions that are contained in C standard library functions

itoa()

- char* itoa(int num, char* str, int base)

- Author: Tanner Forbes

- This function converts an integer literal into an ASCII character based on the ASCII table of characters and values

- Parameters:
  - int num is a number to be converted to a character
  - char* str is a where the new character digit will be stored
  - int base is the base of the number system used

- Return Values:
  - char* is the newly converted digit as a character

bcdToChar()

- char bcdToChar(unsigned char bcd)

- Author: Tanner Forbes

- This function converts a Binary Coded Decimal (BCD) value to a character

- Parameters:
  - unsigned char bcd is the Binary Coded Decimal value to be turned into a character

- Return Values:

- Char is the newly converted BCD as a character

intToBCD()

- unsigned int intToBCD(unsigned int num)

- Author: Chris Jones

- This function converts an integer to a Binary Coded Decimal (BCD) value

- Parameters:
  - unsigned int num is the number to convert to BCD.

- Return Values:
  - Unsigned int is the BCD value of the passed unsigned integer

include/stdlib.h

This file contains a subset of C standard functions

Function Prototypes:

int atoi(const char *s)

- This function converts an ASCII string to an integer literal

char* itoa(int num, char* str, int base)

- This function converts a given integer value into an ASCII string

char bcdToChar(unsigned char bcd)

- This function converts a Binary Coded Decimal Value (BCD) into a character

unsigned int intToBCD(unsigned int num)

- This function converts a character into a Binary Coded Decimal Value (BCD)

include/ctype.h

This file contains a subset of C standard function

Function Prototypes:
int isspace(int c)

- This function determines if the given character is a whitespace

int isdigit(char c)

- This function determines of a given character is a digit

string.c

- string.c holds functions relating to the manipulation or extraction of data of strings.

contains()

- int contains(const char *str1, const char *str2)

- Authors: Tanner Forbes and Chris Jones

- Checks to see if a string is inside of another string

- Parameters:
  - const char *s1 is the phrase to check inside of
  - const char *s2 is the string to find

- Return Values:
  - int will be 1 if the second string was located inside the first and 0 if the string was not found or contained in the first.

substr()

- char* substr(char *str,int index)

- Author: Chris Jones

- This function obtains the substring of a string starting at the passed index and proceeding to the end of the string

- Parameters:
  - char *str is the passed string
  - int index is the index to begin the substring

- Return Values:
  - Char * is the substring starting at the passed index and ending with the last character of the string

isNumeric()

- int isNumeric(char *str)

- Author: Chris Jones

- Checks to see if the passed string is an assortment of only digit character

- Parameters:
  - char *str is the string to check for digits

- Return Values:
  - int will be 1 if the passed string is a digit and 0 if any character is not a digit

charCount()

- int charCount(char str, char c)

- Author: Chris Jones

- Counts the number of occurrences of the character c in a string

- Parameters:
  - char *str is the string of characters to count
  - char c is the character to count at each occurrence in the string

- Return Values:
  - int is the number of occurrences of the character in the string

indexOf()

- int indexOf(char *str, char searched_char)

- Author: Tanner Forbes

- Attempts to find the index of a specified character in a string.

- Parameters:
    - char *str is the string to check for the characters position
    - char searched_char is the character to find the position of

- Return Values:
    - int is the index of the found character or -1 for the character not being in the string

print()
- void print(const char* sentence)

- Author: Izaak Whetsell

- Prints a desired string to the terminal

- Parameters:
    - const char* sentence is the string to be printed in the terminal

- Return Values:
    - There are no return values

println()
- void println(void)

- Author: Izaak Whetsell

- Prints a newline to the terminal

- Parameters:
  - There are no parameters for this function

- Return Values:
  - There are no return values

print_error()

- void print_error(const char* sentence)

- Author: Izaak Whetsell

- Prints the text in the terminal as red to indicate an error has occurred.

- Parameters:
  - const char* sentence is the string to be printed in the terminal

- Return Values:
  - There are no return values

print_color()

- void print_color(const char* sentence, const char* color_code)

- Author: Evan Humphrey

- Prints a desired string to the terminal in the color specified. Available options are green, magenta, blue, yellow, and cyan

- Parameters:

        - const char* sentence is the string to be printed in the terminal
        - const char* color_code is the identifier of what color to print to the terminal.

    - Return Values:
        - There are no return values

trim()

    - void trim (char sentence[])

    - Author: Izaak Whetsell

    - This function trims the whitespace off the given string on each end

    - Parameters:
        - char sentence[] is the string to be trimmed

    - Return Values:
        - Returns the pointer to the trimmed string

user_pcb.c

- user_pcb.c contains all functions that user functions rely on to create, edit, and display process information.

create_PCB()

- int create_PCB(char* name, int class, int priority)

- Author: Chris Jones

- Creates a process control block by allocating memory and setting the fields to the provided parameters.

- Parameters
  - char* name is the name set in the created PCB
  - int class is the class that the PCB will be set to (0 for a user process, 1 for a system process)
  - int priority is the priority number to be assigned to the PCB (process priority is within the range of 0-9, highest to lowest, respectively)

- Return Values:
  - int is the status code of the operation, 0 on success and -1 for an invalid state

delete_PCB()

- int delete_PCB(char* name)

- Author: Chris Jones

- Deletes a specified process control block from memory.

- Parameters
  - char* name is the name to search for in the process queue to delete

- Return Values:
    - int is the status code of the operation, 0 on success and -1 for an invalid state

block_PCB()

- int block_PCB(char* name)

- Author: Chris Jones

- Sets the state of the desired process into a blocked state from the current state it is in, if the process is already blocked, the user will be notified and nothing will be changed.

- Parameters
    - char* name is the name to search for in the process queue to edit

- Return Values:
    - int is the status code of the operation, 0 on success and -1 for an invalid state

unblock_PCB()

- int unblock_PCB(char* name)

- Author: Chris Jones

- Sets the state of the desired process into an unblocked state from the current state it is in, if the process is already unblocked, the user will be notified and nothing will be changed.

- Parameters
    - char* name is the name to search for in the process queue to edit

- Return Values:
  - int is the status code of the operation, 0 on success and -1 for an invalid state

suspend_PCB()

- int suspend_PCB(char* name)

- Author: Chris Jones

- Sets the state of the desired process into a suspended state from the current state it is in, if the process is already suspended, the user will be notified and nothing will be changed.

- Parameters
  - char* name is the name to search for in the process queue to edit

- Return Values:
  - int is the status code of the operation, 0 on success and -1 for an invalid state

resume_PCB()

- int resume_PCB(char* name)

- Author: Chris Jones

- Sets the state of the desired process into a ready state from the current state it is in, if the process is already ready, the user will be notified and nothing will be changed.

- Parameters

        - char* name is the name to search for in the process queue to edit

    - Return Values:
        - int is the status code of the operation, 0 on success and -1 for an invalid state

set_PCB_priority()

    - int set_PCB_priority(char* name, int priority)

    - Author: Chris Jones

    - Sets the priority of the desired process to the given priority.

    - Parameters
        - char* name is the name set in the created PCB
        - int priority is the priority number to be assigned to the PCB (process priority is within the range of 0-9, highest to lowest, respectively)

    - Return Values:
        - int is the status code of the operation, 0 on success and -1 for an invalid state

show_PCB()

    - int show_PCB(char* name)

    - Author: Chris Jones

    - Displays the desired PCB and all of the information contained within it.

    - Parameters

        - char* name is the name to search for in the process queue to display

    - Return Values:
        - int is the status code of the operation, 0 on success and -1 for an invalid state

show_ready_PCB()

- int show_ready_PCB(void)

- Author: Chris Jones

- Displays the ready queue and all processes within it, as well as all of the information contained in each respective process

- Parameters
    - This function has no parameters

- Return Values:
    - int is the status code of the operation, 0 on success and -1 for an invalid state

show_blocked_PCB()

- int show_blocked_PCB(void)

- Author: Chris Jones

- Displays the blocked queue and all processes within it, as well as all of the information contained in each respective process

- Parameters
    - This function has no parameters

- Return Values:
  - int is the status code of the operation, 0 on success and -1 for an invalid state

show_all_PCB()

- int show_all_PCB(void)

- Author: Chris Jones

- Displays the blocked and ready queues and all processes within them, as well as all of the information contained in each respective process

- Parameters
  - This function has no parameters

- Return Values:
  - int is the status code of the operation, 0 on success and -1 for an invalid state

Load_R3()

- int Load_R3(void)

- Author: Izaak Whetsell

- Creates 5 new test rpocesses to test the dispatching implementation of the R3 module of MPX.

- Parameters
  - This function has no parameters

- Return Values:
  - int is the status code of the operation, 0 on success and -1 for an invalid state

Load_R3_Sus()

- int Load_R3_Sus(int pcb_num, int priority)

- Author: Izaak Whetsell

- Creates a ready, suspended process and places it into the suspended queue in the system.

- Parameters
  - int pcb_num is the numbered name of the process to be created

- Return Values:
  - int is the status code of the operation, 0 on success and -1 for an invalid state

includes/mpx/pcb.h

This file contains all of the kernel functions relating to process control blocks

Enums:
pcb_state
READY_SUS,
READY_NOT_SUS,
RUNNNING,
BLOCKED_SUS,
BLOCKED

This enumeration defines all of the possible class and state combinations for processes

Structs:

- **pcb**
  - char* name

        - int class
        - int priority
        - int state
        - unisgned char* stack
        - unsigned char* stack_location
        - struct pcb* next_node
        - struct pcb* prev_node

    - **queue**
        - int queue_type
        - pcb* head
        - pcb* tail

Function Prototypes:
queue* return_queue(int class)
    - This function returns a pointer to the desired queue head

pcb* pcb_setup(char* name, int class, int priority)
    - This function allocates memory for a new PCB and sets all of the required fields within the structure

pcb* pcb_allocate(void)
    - This function dynamically allocates memory for a new PCB struct

int pcb_free(struct pcb* free_pcb)
    - This function frees up the memory allocated to a process control block

pcb* pcb_find(const char * name)
    - This function finds a given process in the system in either queue

void pcb_insert(pcb* pcbPtr)

- This function inserts a new PCB into its corresponding queue in the system

int pcb_remove(pcb* pcbPtr)
- This function removes the specified PCB from the queue but does not delete it from the system

includes/mpx/user_pcb.h

This file contains all user functions related to the creation, alteration, deletion, and display of all process control blocks.

Function Prototypes:
int create_PCB(char* name, int class, int priority)
- This function creates a new process and assigns values into it based on its given parameters

int delete_PCB(char* name)
- This function deletes a PCB from memory

int block_PCB(char* name)
- This function edits a given process to be put into a blocked state

int unblock_PCB(char* name)
- This function edits a given process to be placed into a ready state

int suspend_PCB(char* name)
- This function edits a given process to be placed into a suspended state

int resume_PCB(char* name)
- This function edits a given process to be placed into a not suspended state

int set_PCB_priority(char* name, int priority)

- **This function takes a priority and edits a given process to change to the specified priority**

int show_PCB(char* name)

- **This function displays a given process block and its information given its name**

int show_ready_PCB(void)

- **This function displays all ready processes and their associated information into the terminal window**

int show_blocked_PCB(void)

- **This function displays all blocked processes and their associated information into the terminal window**

int show_all_PCB(void)

- **This function displays all existing processes from the system into the terminal window**

int Load_R3(void)

- **This function deploys 5 new processes to test the dispatching method of the R3 module implementation.**

int Load_R3_Sus(int pcb_num, int priority)

- **This function loads a ready, suspended process into the system with a numbered name specified in the function.**

includes/sys_req.h

This file contains all system request functions and constants

Enums:
op_code

EXIT,
IDLE,
READ,
WRITE

This enumeration defines the constants needed to make a system request to perform a specific action

Constants:

- **INVALID_OPERATION**
- **INVALID_BUFFER**
- **INVALID_COUNT**

These constants define some typical errors and return values that might arise with system calls.

Function Prototypes:
int sys_req(op_code op, ...)

- **This function requests a kernel operation within the system to perform an action based on the input parameters.**

includes/stdlib.h

This file contains a subset of C standard functions

Function Prototypes:
int atoi(const char *s)

- **This function converts an ASCII string to an integer literal**

char* itoa(int num, char* str, int base)

- **This function converts a given integer value into an ASCII string**

char bcdToChar(unsigned char bcd)

- This function converts a Binary Coded Decimal Value (BCD) into a character

unsigned int intToBCD(unsigned int num)

This function converts a character into a Binary Coded Decimal Value (BCD)

includes/string.h

This file contains all functions pertaining to string manipulation and identification

Function Prototypes:
void* memcpy(void * restrict dst, const void * restrict src, size_t n)

- This function will copy a desired region of memory to a destination region in memory and will return a pointer to the destination region

void* memset(void *address, int c, size_t n)

- This function will take a region of memory and fill it with a desired amount of memory and return a pointer to that memory

int strcmp(const char *s1, const char *s2)

- This function compares two strings and returns an integer code specifying the result of the operation

size_t strlen(const char *s)

- This function returns the length of a passed-in string

char* strtok(char * restrict s1, const char * restrict s2)

- This function will tokenize a string and return

int contains(const char *str1, const char *str2)

- This function indicates whether str1 contains str2

char* substr(char* str1, int index)

- This function will return the substring of a passed-in string beginning at a starting index

int isNumeric(char* str)

- This function determines whether a string is entirely numeric

int charCount(char* str, char c)

- This function finds the number of occurrences of a specific character in a given string

int indexOf(char* str, char searched_char)

- This function finds the specific index of a desired character in a string

void print(const char* sentence)

- This function prints out the given sentence into the terminal window

void println(void)
- This function prints a newline to the terminal

void print_error(const char* sentence)
- This function prints an error into the terminal window, this means that the text is printed out in a red color

void print_color(const char* sentence, const char* color_code)
- This function prints out text into the terminal into a variety of colors

void trim(char sentence[])
- This function trims the whitespace off the ends of the given string

time.c

- time.c contains all functions that can retrieve or manipulate time/date functions within the system using the real-time clock (RTC)

get_time()

- int get_time(void)

- Author: Tanner Forbes

- This function utilizes the real time clock registers to display the current time

- Parameters:
  - None

- Return Values:
  - This function returns an integer that represents the status of the functions completion.
  - 1 is to shutdown the operating system
  - 0 is to continue running

set_time()

- int set_time(char buf[])

- Author: Chris Jones

- This function utilizes the real time clock registers to set the current time in the system

- Parameters:
  - char buf[] is an array of characters that the user has typed. This array should contain the proper time format (HH:MM:SS)

- Valid Parameters:
    - Valid hour values include: 01-24
    - Valid minute values include: 00-59
    - Valid second values include: 00-59
- All valid entries for single digits MUST include a beginning 0 before the parameter

- Return Values:
    - This function returns an integer that represents the status of the functions completion.
    - 1 is to shutdown the operating system
    - 0 is to continue running

get_date()
- int get_date(void)

- Author: Tanner Forbes

- This function utilizes the real time clock registers to retrieve and display the current date

- Parameters:
    - None

- Return Values:
    - This function returns an integer that represents the status of the functions completion.
    - 1 is to shutdown the operating system
    - 0 is to continue running

set_date()
- int set_date(char buf[])

- Author: Tanner Forbes

- This function utilizes the real time clock registers to set the current date in the system

- Parameters:
  - char buf[] is an array of characters that the user has typed. This array should contain the proper date format (MM/DD/YYYY) and should accurately match a calendar.
  - Example requirements include no months greater than 12, no days greater than 31, each month has designated 28-30-31 day scheme, leap year, and year is within the 21st Century.

- Return Values:
  - This function returns an integer that represents the status of the functions completion.
  - 1 is to shutdown the operating system
  - 0 is to continue running

get_hour()

- int get_hour(void)

- Author: Tanner Forbes

- This function utilizes the real time clock registers to return the current hour

- Parameters:
  - None

- Return Values:
  - This function returns an integer representation of the current hour in the system.

get_minute()

- int get_minute(void)

- Author: Tanner Forbes

- This function utilizes the real time clock registers to return the current minute

- Parameters:
  - None

- Return Values:
  - This function returns an integer representation of the current minute in the system.

get_second()
- int get_second(void)

- Author: Tanner Forbes

- This function utilizes the real time clock registers to return the current second

- Parameters:
  - None

- Return Values:
  - This function returns an integer representation of the current second in the system.

includes/sys_req.h

This file contains all system request functions and constants

Enums:
op_code

EXIT,
IDLE,
READ,
WRITE

This enumeration defines the constants needed to make a system request to perform a specific action

Constants:

- **INVALID_OPERATION**
- **INVALID_BUFFER**
- **INVALID_COUNT**

These constants define some typical errors and return values that might arise with system calls.

Function Prototypes:
int sys_req(op_code op, ...)

- **This function requests a kernel operation within the system to perform an action based on the input parameters.**

includes/stdlib.h

This file contains a subset of C standard functions

Function Prototypes:
int atoi(const char *s)

- **This function converts an ASCII string to an integer literal**

char* itoa(int num, char* str, int base)

- This function converts a given integer value into an ASCII string

char bcdToChar(unsigned char bcd)

- This function converts a Binary Coded Decimal Value (BCD) into a character

unsigned int intToBCD(unsigned int num)

This function converts a character into a Binary Coded Decimal Value (BCD)

includes/mpx/io.h

This header file contains kernel macros to write and read to I/O ports

Function Prototypes:
outb(port, data)

- This will take one byte of data and write it to an I/O port

inb(port, data)

- This will take one byte of data and read it into an I/O port

includes/string.h
This file contains all functions pertaining to string manipulation and identification

Function Prototypes:
void* memcpy(void * restrict dst, const void * restrict src, size_t n)

- This function will copy a desired region of memory to a destination region in memory and will return a pointer to the destination region

void* memset(void *address, int c, size_t n)

- This function will take a region of memory and fill it with a desired amount of memory and return a pointer to that memory

int strcmp(const char *s1, const char *s2)

- This function compares two strings and returns an integer code specifying the result of the operation

size_t strlen(const char *s)

- This function returns the length of a passed-in string

char* strtok(char * restrict s1, const char * restrict s2)

- This function will tokenize a string and return

int contains(const char *str1, const char *str2)

- This function indicates whether str1 contains str2

char* substr(char* str1, int index)

- This function will return the substring of a passed-in string beginning at a starting index

int isNumeric(char* str)

- This function determines whether a string is entirely numeric

int charCount(char* str, char c)

- This function finds the number of occurrences of a specific character in a given string

int indexOf(char* str, char searched_char)

- This function finds the specific index of a desired character in a string

void print(const char* sentence)
- This function prints out the given sentence into the terminal window

void println(void)
- This function prints a newline to the terminal

void print_error(const char* sentence)
- This function prints an error into the terminal window, this means that the text is printed out in a red color

void print_color(const char* sentence, const char* color_code)
- This function prints out text into the terminal into a variety of colors

void trim(char sentence[])
This function trims the whitespace off the ends of the given string

includes/mpx/interrupts.h

This file contains kernel functions related to software and hardware interrupts in the system

Constants include:

- **cli()**
- **sti()**

These constants are used to disable and enable interrupts in the system respectively

Function Prototypes:
void irq_init(void)

- **This function installs the first 32 interrupt handlers into the IRQ (Interrupt Request Table)**

void pic_init(void)

- **This function initializes the programmable interrupt controllers and performs the necessary remapping of IRQs, it leaves interrupts turned off.**

void idt_init(void)

- **This function creates and installs the interrupt descriptor table**

void idt_install(int vector, void (*handler)(void *))

- **The function installs an interrupt handler**

include/stdlib.h

This file contains a subset of C standard functions

Function Prototypes:

int atoi(const char *s)

- This function converts an ASCII string to an integer literal

char* itoa(int num, char* str, int base)

- This function converts a given integer value into an ASCII string

char bcdToChar(unsigned char bcd)

- This function converts a Binary Coded Decimal Value (BCD) into a character

unsigned int intToBCD(unsigned int num)

- This function converts a character into a Binary Coded Decimal Value (BCD)

pcb.c

- pcb.c contains all kernel level functions pertaining to the manipulation of process control blocks and queues

- Queues:
  - BLOCKED
    - int queue_type: 1 (blocked)
    - pcb* head: NULL
    - pcb* tail: NULL
  - READY
    - int queue_type: 0 (ready)
    - pcb* head: NULL
    - pcb* tail: NULL

return_queue()

- queue* return_queue(int class)

- Author: Chris Jones

- This function returns a pointer to the head of the specified queue

- Parameters:
  - int class specifies what queue we would like to return

- Return Values:
  - This function returns a pointer to the desired queue

pcb_setup()

- pcb* pcb_setup(char* name, int class, int priority, int state, void (*function_ptr)(void))

- Author: Chris Jones & Izaak Whetsell

- This function allocates memory for a new process and sets each of its fields according to the specified parameters

- Parameters:
  - char* name is the name of the process
  - int class identifies the process as ready or blocked and suspended or not suspended
  - int priority sets the priority number of the process from highest to lowest (0-9)
  - void* function_ptr is the pointer to the process running that will be stored into it's context

- Return Values:
  - This function returns a pointer to the newly created process

pcb_allocate()

- pcb* pcb_allocate(void)

- Author: Tanner Forbes

- This function dynamically allocates memory for a new process and returns a pointer to it

- Parameters:
  - There are no parameters for this function

- Return Values:
  - This function returns a pointer to the dynamically allocated PCB

pcb_free()

- int pcb_free(struct pcb* free_pcb)

- Author: Tanner Forbes

- This function frees the memory used by a PCB

- Parameters:
  - Struct pcb* free_pcb is the pointer to the point in memory where we would like to free

- Return Values:
  - This function returns a status code based on the operation performed
    - 0 for success
    - 1 for an error

pcb_find()

- pcb* pcb_find(const char * name)

- Author: Tanner Forbes

- This function searches for a specific PCB in queue based on its name parameter

- Parameters:
  - const char* name is the name of the PCB we would like to find

- Return Values:
  - This function returns a pointer to the located PCB

pcb_insert()

- void pcb_insert(pcb* pcbPtr)

- Author: Izaak Whetsell

- This function inserts a PCB into its defined queue

- Parameters:
  - pcb* pcbPtr is the pointer to the PCB we would like to insert into a queue

- Return Values:
  - This function has no return values

pcb_remove()

- int pcb_remove(pcb* pcbPtr)

- Author: Tanner Forbes

- This function removes a PCB from the queue but does not delete it from the system.

- Parameters:
  - pcb* pcbPtr is the pointer to the PCB we would like to remove from the queue

- Return Values:
  - This function returns a status code based on the operation
    - 0 for success
    - 1 for an error

includes/mpx/pcb.h

This file contains all of the kernel functions relating to process control blocks

Enums:
pcb_state
READY_SUS,
READY_NOT_SUS,
RUNNNING,

BLOCKED_SUS,
BLOCKED

This enumeration defines all of the possible class and state combinations for processes

Structs:
- **pcb**
  - char* name
  - int class
  - int priority
  - int state
  - unisgned char* stack
  - unsigned char* stack_location
  - struct pcb* next_node
  - struct pcb* prev_node

- **queue**
  - int queue_type
  - pcb* head
  - pcb* tail

Function Prototypes:
queue* return_queue(int class)
- This function returns a pointer to the desired queue head

pcb* pcb_setup(char* name, int class, int priority)
- This function allocates memory for a new PCB and sets all of the required fields within the structure

pcb* pcb_allocate(void)
- This function dynamically allocates memory for a new PCB struct

int pcb_free(struct pcb* free_pcb)

- This function frees up the memory allocated to a process control block

pcb* pcb_find(const char * name)

- This function finds a given process in the system in either queue

void pcb_insert(pcb* pcbPtr)

- This function inserts a new PCB into its corresponding queue in the system

int pcb_remove(pcb* pcbPtr)
This function removes the specified PCB from the queue but does not delete it from the system

includes/memory.h

This file contains system-specific dynamic memory functions

Function Prototypes:
void *sys_alloc_mem(size_t size)

- This function dynamically allocates memory in the system

int sys_free_mem(void *ptr)

- This function frees dynamically allocated memory located within the system

void sys_set_heap_functions(void * (*alloc_fn)(size_t), int (*free_fn)(void *))

This function installs user-supplied heap-management functions

includes/string.h

This file contains all functions pertaining to string manipulation and identification

int contains(const char *str1, const char *str2)

- This function indicates whether str1 contains str2

char* substr(char* str1, int index)

- This function will return the substring of a passed-in string beginning at a starting index

int isNumeric(char* str)

- This function determines whether a string is entirely numeric

int charCount(char* str, char c)

- This function finds the number of occurrences of a specific character in a given string

int indexOf(char* str, char searched_char)

- This function finds the specific index of a desired character in a string

void print(const char* sentence)

- This function prints out the given sentence into the terminal window

void println(void)

- This function prints a newline to the terminal

void print_error(const char* sentence)

- This function prints an error into the terminal window, this means that the text is printed out in a red color

void print_color(const char* sentence, const char* color_code)

- This function prints out text into the terminal into a variety of colors

void trim(char sentence[])

- This function trims the whitespace off the ends of the given string

<u>includes/context.h</u>

This file contains all structures and functions relating to the context of a process.

Structs:

- **context**
  - Segment Registers:
    - int CS
    - int DS
    - int ES
    - int FS
    - int GS
    - int SS
  - Status Control Registers:
    - int EIP
    - int EFLAGS
  - General Purpose Registers:
    - int EAX
    - int EBX
    - int ECX
    - int EDX
    - int ESI

- int EDI
- int EBP
- int ESP

sys_call.c

- sys_call.c contains all functions pertaining to dispatching in the OS. These functions alter the currently running processes and queues.

sys_call()

- struct context* sys_call(context* ctx)

- Author: Chris Jones, Evan Humphrey

- This function takes the currently running process in the queue and performs the specified operation.

- Parameters:
  - context ctx* is the pointer to the context of the next process to be selected from the queue

- Return Values:
  - This function returns a pointer to the context of the next process in the queue.

find_first_ready()

- pcb* find_first_ready(void)

- Author: Chris Jones

- This function finds the first process in the ready queue and returns a pointer to it.

- Parameters:
  - There are no parameters for this function

- Return Values:
  - This function returns a pointer to the first PCB in the ready queue.

includes/mpx/sys_call.h

This file contains all functions pertaining to process dispatching.

Global Variables:
pcb* CURRENT_PCB

This pointer points to the currently running process in the system, and is updated during every context switch.

ctx* GLOBAL_CTX

This pointer points to the context of the initial process loaded in the system, and is used to return back to the system's initial state when an *EXIT* is issued.

pcb* nextPCB

This pointer is used to track the position of the next process in the queue.

struct context* sys_call(context* ctx)
- This function returns a pointer to the next process to execute in the system.

pcb* find_first_ready(void)
- This functions finds and returns the first process in the ready queue.

includes/context.h

This file contains all structures and functions relating to the context of a process.

Structs:

- **context**
  - Segment Registers:
    - int CS
    - int DS
    - int ES
    - int FS
    - int GS
    - int SS
  - Status Control Registers:
    - int EIP
    - int EFLAGS
  - General Purpose Registers:
    - int EAX
    - int EBX
    - int ECX
    - int EDX
    - int ESI
    - int EDI
    - int EBP
    - int ESP

includes/mpx/pcb.h

This file contains all of the kernel functions relating to process control blocks

Enums:
pcb_state
READY_SUS,
READY_NOT_SUS,
RUNNNING,
BLOCKED_SUS,
BLOCKED

This enumeration defines all of the possible class and state combinations for processes

Structs:

- **pcb**
  - char* name
  - int class
  - int priority
  - int state
  - unisgned char* stack
  - unsigned char* stack_location
  - struct pcb* next_node
  - struct pcb* prev_node

- **queue**
  - int queue_type
  - pcb* head
  - pcb* tail

Function Prototypes:

queue* return_queue(int class)

- This function returns a pointer to the desired queue head

pcb* pcb_setup(char* name, int class, int priority)

- This function allocates memory for a new PCB and sets all of the required fields within the structure

pcb* pcb_allocate(void)

- This function dynamically allocates memory for a new PCB struct

int pcb_free(struct pcb* free_pcb)

- This function frees up the memory allocated to a process control block

```
pcb* pcb_find(const char * name)
  - This function finds a given process in the system in either
    queue

void pcb_insert(pcb* pcbPtr)
  - This function inserts a new PCB into its corresponding
    queue in the system

int pcb_remove(pcb* pcbPtr)
  - This function removes the specified PCB from the queue but
    does not delete it from the system
```

alarm.c

- alarm.c contains all functions pertaining to alarm dispatch and monitoring.

alarm_process()

- void alarm_process(void)

- Author: Chris Jones

- This function monitors the currently running alarms in the queue and makes sure they trigger when needed.

- Parameters:
  - None

- Return Values:
  - This function has no return values

create_alarm()

- int create_alarm(char* message, char* time)

- Author: Chris Jones

- This function takes parameters from the user and creates an alarm process that is placed into the process queue

- Parameters:
  - message is the message to be displayed to the user when the alarm is triggered
  - time is the system time specified by the user for when the alarm should trigger

- Return Values:
  - This function returns a status code based on the operation

- -1 for failure
- 0 for success

include/mpx/time.h

This file contains all functions relating to the real time clock (RTC)

Function Prototypes:
int get_time(void)
- This function prints out the current time in Eastern Standard Time using the real time clock

int set_time(char buf[])
- This function utilizes the real time clock registers to set the time in the system

int get_date(void)
- This function retrieves the date using the real time clock and prints it to the terminal

int set_date(char buf[])
- This function utilizes the real time clock registers to set the current date within the system

int get_hour(void)
- This function returns an integer representation of the current hour in the system.

int get_minute(void)
- This function returns an integer representation of the current minute in the system.

int get_second(void)

- This function returns an integer representation of the current second in the system.

includes/mpx/pcb.h

This file contains all of the kernel functions relating to process control blocks

Enums:
pcb_state
READY_SUS,
READY_NOT_SUS,
RUNNNING,
BLOCKED_SUS,
BLOCKED

This enumeration defines all of the possible class and state combinations for processes

Structs:
- **pcb**
    - char* name
    - int class
    - int priority
    - int state
    - unisgned char* stack
    - unsigned char* stack_location
    - struct pcb* next_node
    - struct pcb* prev_node

- **queue**
    - int queue_type
    - pcb* head
    - pcb* tail

Function Prototypes:
queue* return_queue(int class)
- This function returns a pointer to the desired queue head

pcb* pcb_setup(char* name, int class, int priority)
- This function allocates memory for a new PCB and sets all of the required fields within the structure

pcb* pcb_allocate(void)
- This function dynamically allocates memory for a new PCB struct

int pcb_free(struct pcb* free_pcb)
- This function frees up the memory allocated to a process control block

pcb* pcb_find(const char * name)
- This function finds a given process in the system in either queue

void pcb_insert(pcb* pcbPtr)
- This function inserts a new PCB into its corresponding queue in the system

int pcb_remove(pcb* pcbPtr)
- This function removes the specified PCB from the queue but does not delete it from the system

include/sys_req.h

This file contains all system request functions and constants

Enums:
op_code
EXIT,

IDLE,
READ,
WRITE

This enumeration defines the constants needed to make a system request to perform a specific action

Constants:

- **INVALID_OPERATION**
- **INVALID_BUFFER**
- **INVALID_COUNT**

These constants define some typical errors and return values that might arise with system calls.

Function Prototypes:
int sys_req(op_code op, ...)

- This function requests a kernel operation within the system to perform an action based on the input parameters.

include/string.h

This file contains all functions pertaining to string manipulation and identification

int contains(const char *str1, const char *str2)

- This function indicates whether str1 contains str2

char* substr(char* str1, int index)

- This function will return the substring of a passed-in string beginning at a starting index

int isNumeric(char* str)

- This function determines whether a string is entirely numeric

int charCount(char* str, char c)

- This function finds the number of occurrences of a specific character in a given string

int indexOf(char* str, char searched_char)

- This function finds the specific index of a desired character in a string

void print(const char* sentence)

- This function prints out the given sentence into the terminal window

void println(void)

- This function prints a newline to the terminal

void print_error(const char* sentence)

- This function prints an error into the terminal window, this means that the text is printed out in a red color

void print_color(const char* sentence, const char* color_code)

- This function prints out text into the terminal into a variety of colors

void trim(char sentence[])

- This function trims the whitespace off the ends of the given string

includes/mpx/sys_call.h

This file contains all functions pertaining to process dispatching.

Global Variables:
pcb* CURRENT_PCB

This pointer points to the currently running process in the system, and is updated during every context switch.

ctx* GLOBAL_CTX

This pointer points to the context of the initial process loaded in the system, and is used to return back to the system's initial state when an *EXIT* is issued.

pcb* nextPCB

This pointer is used to track the position of the next process in the queue.

struct context* sys_call(context* ctx)
- This function returns a pointer to the next process to execute in the system.

pcb* find_first_ready(void)
- This functions finds and returns the first process in the ready queue.

sys_call_isr.s

This file contains the code that performs the context switch of a process by using assembly to retrieve and store the values of currently running processes.